PumpGuard
(Pumpfun-AI-Attack-Defence)

Project Requirements Document

UE5 / Unity Game Platform Edition

Version: 2.1 Date: March 24, 2026 Classification: Confidential

1. Executive Summary

Pumpfun-AI-Attack-Defence is a next-generation platform sitting at the intersection of Artificial Intelligence, Cryptocurrency, and Gaming — built natively on the Solana blockchain. The project combines Pump.fun's proven token-launch bonding curve mechanics, Meteora's dynamic liquidity infrastructure, and autonomous AI agents into a competitive attack-defence gaming ecosystem.

Players deploy AI-driven agents that either attack (exploit pricing inefficiencies, execute pump strategies, simulate adversarial market behavior) or defend (detect manipulation, stabilize liquidity, counter sniper-bot activity) across live or simulated Pump.fun token environments. The result is a gamified security platform that stress-tests DeFi protocols while rewarding participants for improving ecosystem resilience.

▸ What's New in v2.0 — UE5 / Unity Game Platform

This revision upgrades the game presentation and interaction layer from a web-based React dashboard to a full 3D real-time game client built with Unreal Engine 5 (UE5) and/or Unity. The goal is to deliver a visually stunning, AAA-quality game experience — featuring cinematic 3D arenas, real-time particle-driven attack/defence visualizations, immersive spectator cameras, and rich spatial audio — while retaining the existing Solana smart contract layer, AI engine, and protocol integrations unchanged.

2. Background & Market Context

2.1 Pump.fun Ecosystem

Pump.fun is the dominant memecoin launchpad on Solana, having facilitated over 1 million token creations and generating $200M+ in cumulative revenue. It employs a constant-product linear bonding curve ($x \cdot y = k$) for fair price discovery — token price increases proportionally with circulating supply, incentivizing early participation. When a token's bonding curve reaches approximately 85 SOL in accumulated liquidity, it automatically migrates to a DEX (Meteora/Raydium) for open-market trading.

2.2 Meteora & Liquidity Infrastructure

Meteora operates as Solana's advanced liquidity layer, providing Dynamic Liquidity Market Maker (DLMM) pools. Upon Pump.fun migration, tokens transition from the bonding curve into Meteora's concentrated liquidity pools. This migration event is a critical attack surface — sniper bots and MEV extractors frequently exploit the transition window.

2.3 The Bot Arms Race

The Pump.fun ecosystem is plagued by automated bots: snipers that front-run token launches, bundlers that coordinate multi-wallet pump schemes, and rug-pull bots that drain liquidity post-

migration. Current defenses are reactive and manual. This project transforms that dynamic into a competitive, AI-driven game.

2.4 Why UE5 / Unity for a Web3 Game?

Traditional Web3 gaming platforms rely on browser-based interfaces that limit visual fidelity, interactivity, and player immersion. By adopting Unreal Engine 5 or Unity as the game client, we unlock:

- Nanite & Lumen (UE5) or URP/HDRP (Unity) for photorealistic arena rendering
- Real-time physics and particle systems to visualize on-chain attack/defence events
- Cross-platform deployment: PC, Mac, consoles, mobile, and WebGL/WASM for browser fallback
- Rich multiplayer networking with dedicated game servers
- AAA-quality spectator mode with cinematic replay systems
- Seamless wallet integration via embedded browser widgets or native Solana SDKs

3. Technical Architecture

3.1 System Layers (Updated)

Layer	Description	Technology
L1 — Blockchain	On-chain smart contracts, bonding curve logic, token program	Solana, Anchor Framework (Rust)
L2 — Protocol	Pump.fun integration, Meteora DLMM migration, fee routing	FunPump SDK, Meteora SDK
L3 — AI Engine	Agent orchestration, LLM strategy generation, ML anomaly detection	Python/TypeScript, LLM APIs, TensorFlow/PyTorch
L4 — Game Server	Dedicated game server: matchmaking, game state sync, scoring, leaderboards, relay between blockchain events and game client	UE5 Dedicated Server / Unity Netcode for GameObjects, WebSocket bridge, Redis, PostgreSQL
L5 — Game Client (UE5/Unity)	3D real-time game client: arena visualization, agent deployment UI, spectator cam, wallet connection, live analytics HUD	Unreal Engine 5 (C++ / Blueprints) OR Unity (C#), Solana wallet SDK, REST/WebSocket to L4
L6 — Web Companion (Optional)	Lightweight browser dashboard for leaderboards, account management, NFT marketplace browsing	React / Next.js, D3.js

3.2 Smart Contract Requirements (Unchanged)

Bonding Curve Engine: Fork and extend the Pump.fun smart contract using Anchor (Rust). Implement configurable bonding curve parameters (slope, initial price, migration threshold). Use Program Derived Accounts (PDA) for deterministic state management.

Migration Controller: Automated liquidity migration to Meteora DLMM pools at threshold. Emit on-chain events for AI agent consumption.

Fee & Reward Router: Dynamic fee collection routed to protocol multisig wallet. Reward distribution to winning attack/defence agents per epoch.

Agent Registry: On-chain registry of AI agent identities, performance records, and staked collateral.

3.3 AI Agent Framework (Unchanged)

Offensive Agents (Attack Mode)

- Execute automated pump/dump strategies on sandboxed or live token launches
- Simulate sniper-bot behavior — front-running, sandwich attacks, MEV extraction
- Coordinated multi-agent swarm attacks with adjustable aggression parameters
- Adaptive strategy evolution using reinforcement learning across game rounds

Defensive Agents (Defence Mode)

- Real-time anomaly detection on transaction patterns using ML classifiers (isolation forests, autoencoders)
- Counter-trade execution to neutralize coordinated pump activity
- Automated liquidity injection to stabilize bonding curve pricing
- Prompt injection & adversarial input protection for LLM-controlled agent parameters
- Risk scoring of incoming transactions with configurable alert thresholds

3.4 Game Client Architecture (NEW — UE5/Unity)

3.4.1 Rendering & Visual Design

- **3D Arena Environment:**

Each token arena is rendered as a distinct 3D battleground. The bonding curve is visualized as a towering holographic structure at the arena center — its shape morphing in real-time as price changes. Attack events manifest as projectile volleys, energy beams, and explosive particle bursts; defence events appear as shield barriers, healing waves, and counter-pulse fields.

- **UE5 Specifics:**

Leverage Nanite for highly detailed arena geometry, Lumen for dynamic global illumination, Niagara particle system for attack/defence VFX, MetaHuman-quality agent avatars, and World Partition for streaming large arena maps.

- **Unity Specifics:**

Use HDRP (High Definition Render Pipeline) for console/PC quality, URP (Universal Render Pipeline) for mobile/WebGL, VFX Graph for GPU-accelerated particles, Shader Graph for custom arena materials, and Addressables for asset streaming.

3.4.2 Game Client Modules

Module	Description	Engine Feature
Arena Renderer	Real-time 3D visualization of token bonding curve, price movements, and liquidity pools as interactive environments	UE5: Nanite + Lumen + Niagara Unity: HDRP + VFX Graph
Agent Visualization	Each AI agent rendered as a unique 3D character/mech with attack/defence animations tied to on-chain actions	UE5: Skeletal Mesh + Anim Blueprints Unity: Animator Controller + Timeline
HUD & UI System	In-game overlay: live price chart, agent health/stats, transaction feed, wallet balance, staking controls	UE5: UMG (Unreal Motion Graphics) Unity: UI Toolkit / TextMeshPro
Spectator Camera	Cinematic free-cam, agent-follow cam, arena overview cam with smooth transitions and replay scrubbing	UE5: Sequencer + Camera Rigs Unity: Cinemachine + Timeline
Wallet Integration	Native Solana wallet connection (Phantom, Solflare) via embedded browser widget or native SDK bridge	UE5: Web Browser Widget + C++ bridge Unity: WebView + C# Solana SDK
Audio System	Spatial audio for attack/defence events, dynamic music that intensifies with market volatility, announcer voice for major events	UE5: MetaSounds Unity: FMOD / Wwise integration
Networking Layer	Real-time game state sync with dedicated server, WebSocket bridge to blockchain event stream	UE5: Dedicated Server + Replication Unity: Netcode for GameObjects + Transport

3.4.3 Cross-Platform Build Targets

Platform	Engine	Notes
Windows / Mac / Linux	UE5 or Unity	Primary target — full visual fidelity
WebGL / WASM (Browser)	Unity (WebGL) or UE5 (Pixel Streaming)	Fallback for browser-only users; reduced fidelity
iOS / Android	Unity (URP)	Mobile companion with simplified arena view
PlayStation / Xbox (Future)	UE5 or Unity	Console port for expanded reach

3.4.4 Game Server Architecture

The dedicated game server acts as the authoritative bridge between the blockchain layer and game clients. It is responsible for:

- Subscribing to Solana on-chain events (transactions, state changes) via WebSocket/RPC
- Translating blockchain events into game-world events (attacks, defences, price movements)
- Managing arena lifecycle: creation, agent deployment, live phase, scoring, settlement
- Matchmaking players based on ELO rating, stake amount, and agent tier
- Replicating authoritative game state to all connected game clients
- Recording full match replays for historical analysis and spectator playback
- Horizontal scaling via containerized server instances (Kubernetes / ECS)

Server Technology Options:

- **UE5 Path:**

UE5 Dedicated Server builds (C++) with custom GameMode handling arena logic. Replication system syncs game state to clients. External microservices (Node.js/Python) handle blockchain subscriptions and AI orchestration, communicating via gRPC or REST.

- **Unity Path:**

Unity Netcode for GameObjects with a custom transport layer. Alternatively, use a standalone C#/.NET game server with Unity's serialization. External services for blockchain and AI via REST/WebSocket.

- **Hybrid Path (Recommended):**

A polyglot architecture where the core game server (UE5/Unity) handles rendering-related state and client sync, while a separate backend service layer (Node.js/Python) manages blockchain interactions, AI orchestration, matchmaking queues, and persistent data. Communication between layers via Redis pub/sub + gRPC.

3.4.5 Rendering Pipeline & Performance Budgets

To ensure consistent visual quality and frame-rate stability across all target platforms, the following rendering budgets and pipeline specifications are enforced:

Frame Rate Targets: PC (Recommended) — 60 FPS at 1440p with all effects enabled; PC (Minimum) — 30 FPS at 1080p with reduced LOD and particle density; WebGL/Browser — 30 FPS at 1080p with simplified shaders and capped draw calls; Mobile (Unity URP) — 30 FPS at 720p with baked lighting and sprite-based VFX fallback.

Draw Call Budget: Maximum 2,500 draw calls per frame on PC; 800 on mobile. Achieved via GPU instancing for agent crowds, hierarchical LOD (4-tier) for arena geometry, and texture atlas

consolidation for UI elements. UE5 path leverages Nanite's automatic LOD elimination; Unity path requires manual LOD Group configuration with Impostor generation for distant agents.

Polygon Budget: Arena environment — 2M triangles (Nanite virtualizes to 20M+ source); Agent characters — 50K triangles (LOD0), 15K (LOD1), 5K (LOD2); Bonding curve hologram — 10K triangles + GPU-driven tessellation shader. Total scene budget: 4M visible triangles maximum (PC), 500K (mobile).

Memory Budget: GPU VRAM — 4 GB (minimum), 6 GB (recommended); Texture streaming pool — 2 GB with priority-based eviction; VFX particle pool — 50,000 concurrent GPU particles (PC), 5,000 (mobile); Audio memory — 256 MB for spatial audio banks and dynamic music stems.

Shader Complexity: All arena materials capped at 200 shader instructions (PS). Bonding curve hologram uses a custom multi-pass shader: Pass 1 — wireframe grid with animated UV scroll; Pass 2 — volumetric glow via ray-marched signed distance field; Pass 3 — data-driven color ramp mapped to real-time price delta. Mobile/WebGL fallback replaces volumetric passes with screen-space bloom approximation.

3.4.6 Asset Production Pipeline

A structured art production pipeline ensures visual consistency and efficient iteration across all 3D assets — arenas, agents, VFX, and UI elements.

3D Modeling & Sculpting: High-poly sculpts in ZBrush/Blender → retopology to game-ready meshes → bake normal/AO/curvature maps → PBR texturing in Substance 3D Painter. Agents use modular skeletal mesh architecture: base body + swappable armor/weapon/aura attachments for NFT-driven cosmetic variation. Arena environments built from tileable modular kits (floors, walls, pillars, holo-projectors) for procedural assembly based on token metadata.

Animation Pipeline: Agent locomotion and combat animations authored in Maya/Blender using motion-captured reference data. Attack animations: 8 melee + 6 ranged + 4 special ability variants per agent class. Defence animations: 4 shield-deploy + 3 counter-pulse + 3 healing wave + 2 evasion variants. Idle/celebration/defeat animations for pre/post-match sequences. UE5: Animation Blueprints with state machines and blend spaces; Unity: Animator Controllers with sub-state machines and Avatar Masks for upper/lower body blending.

VFX Asset Specifications: Each attack type maps to a unique VFX template: Pump attack → ascending energy column (GPU particles + mesh distortion); Sniper attack → precision beam with hit-spark burst; Sandwich attack → dual-sided pincer shockwave; Swarm attack → dispersed particle cloud with trail ribbons. Each defence type: Shield → hemispherical force-field with hex-grid pattern and impact ripple; Counter-trade → radial pulse ring; Liquidity injection → upward fountain of glowing orbs; Anomaly detection → scanning radar sweep with highlight markers. All VFX authored in UE5 Niagara / Unity VFX Graph with data-driven parameters (intensity, color, scale) driven by real-time on-chain transaction magnitude.

Texture & Material Standards: All textures authored at 4K source, delivered at 2K (PC) / 1K (mobile) with BC7/ASTC compression. PBR metalness/roughness workflow. Shared material library with 20+ master materials and 100+ material instances. Dynamic Material Parameter Collections drive arena-wide mood shifts (lighting color, fog density, emissive intensity) based on market volatility index.

3.4.7 Networking & Replication Specifications

Real-time multiplayer synchronization between game server and clients requires precise networking specifications to balance responsiveness, bandwidth, and authority.

Server Tick Rate: 30 Hz (33 ms per tick) for arena game state updates. Client render interpolation at 60 Hz with 100 ms interpolation buffer. Server-authoritative model: all scoring, agent state, and transaction validation runs server-side; clients are prediction-only with server reconciliation.

Bandwidth Budget: Per-client downstream — 50 KB/s average, 150 KB/s peak (during swarm attacks). Per-client upstream — 10 KB/s (agent commands + camera state). Achieved via delta compression, quantized vector packing (16-bit positions, 8-bit rotations), and relevancy-based culling (agents/events outside camera frustum are deprioritized).

Blockchain Event Bridge: A dedicated WebSocket relay service subscribes to Solana RPC (confirmed commitment level) and transforms raw transaction data into typed game events. Event types: AgentAction, PriceUpdate, LiquidityChange, MigrationTrigger, AnomalyAlert. Target bridge latency: < 500 ms from on-chain confirmation to game-world visualization. Failover: dual-RPC subscription (primary + backup) with automatic switchover on 3 consecutive timeouts.

Matchmaking & Session Management: ELO-based matchmaking with ±200 rating tolerance, expanding by 50 per 30-second queue cycle. Maximum 20 agents per arena (10 attackers vs 10 defenders). Maximum 500 concurrent spectators per arena via a separate read-only replication channel. Session lifecycle: Lobby (60s) → Deployment (30s) → Live Phase (5–15 min configurable) → Settlement (30s) → Results (60s).

3.4.8 Build Configuration & Platform Packaging

Each platform target requires specific build configurations, asset processing, and QA validation.

Windows (Primary): UE5 — Development/Shipping build configurations; Pak file packaging with AES-256 encryption; DirectX 12 / Vulkan renderer selection at launch; dedicated server build as separate target. Unity — IL2CPP scripting backend; LZ4 asset compression; DirectX 12 with Vulkan fallback.

WebGL / Browser Fallback: Unity WebGL — Brotli-compressed WASM build; 256 MB memory allocation; reduced texture pool (512 MB); URP pipeline with simplified post-processing stack. Target initial load: < 30 seconds on 50 Mbps. UE5 Pixel Streaming — server-rendered frames

streamed via WebRTC; requires GPU compute instances (NVIDIA T4 or better); latency target < 80 ms; auto-scaling via AWS GameLift or Hathora.

Mobile (Future — Unity URP): ARM64 builds for iOS 16+ and Android 12+; Vulkan renderer (Android) / Metal (iOS); Adaptive resolution scaling (50%–100% render scale based on thermal state); Simplified arena view — 2.5D overhead camera with sprite-based agents as optional ultra-low-spec mode. Target app size: < 500 MB initial download + 200 MB on-demand asset packs via Addressables.

CI/CD Pipeline: Automated builds via Jenkins / GitHub Actions with per-platform build agents. UE5: BuildGraph XML scripts for parallel cook + package; Unity: Unity Build Automation (Cloud Build) or custom CLI (unity -batchmode). Automated smoke tests: launch → connect wallet → enter arena → render 60 frames → screenshot comparison. Artifact distribution via Steam (PC), itch.io (early access), TestFlight/Firebase App Distribution (mobile).

4. Functional Requirements

4.1 Core Game Loop

Token Arena Setup — A new Pump.fun token is launched (sandboxed or live) with defined bonding curve parameters. The 3D arena environment is procedurally generated based on token metadata (name, symbol, curve shape).

Agent Deployment — Players deploy their custom AI agents into the arena, choosing Attack or Defence roles. Each agent stakes SOL as collateral. In the game client, agents materialize as 3D characters with role-specific visual themes (red/aggressive for attackers, blue/protective for defenders).

Live Engagement — Agents execute strategies in real-time. On-chain transactions are translated into cinematic 3D combat: attackers launch projectiles at the bonding curve structure, defenders deploy shields and counter-attacks. The arena environment dynamically reacts — lighting shifts, particle intensity changes, and the bonding curve hologram distorts under stress.

Scoring & Settlement — Performance is evaluated on price impact, detection accuracy, response latency, and capital efficiency. Rewards are distributed from the losing side's staked collateral to the winners. A dramatic end-of-match sequence plays showing final scores and reward distribution.

Evolution — Agents retain learned strategies across rounds. Top-performing agents can be minted as NFTs for trading. In the game client, evolved agents gain visual upgrades: glowing auras, enhanced armor, unique VFX trails.

4.2 Player Workflows

Role	Actions

Token Creator	Launch token via AI prompt → Configure bonding curve → Watch arena generate in 3D → Monitor defence performance → Earn creator rewards at $69K market cap
Attacker	Design offensive AI strategy → Customize agent appearance → Stake SOL → Deploy agent into 3D arena → Watch attack unfold in real-time → Earn rewards for successful exploits
Defender	Design detection/counter strategy → Customize agent appearance → Stake SOL → Deploy agent into 3D arena → Watch defence in action → Earn rewards for successful protection
Spectator	Browse live arenas in 3D lobby → Enter arena as observer → Free-cam or follow specific agents → Bet on outcomes → Watch replays with cinematic camera → Analyze agent strategies

4.3 In-Game HUD & Analytics

- Live 3D bonding curve hologram with real-time price, volume, and liquidity overlays
- Agent stat panels: health bar (collateral remaining), attack/defence power, win streak
- Mini-map showing all agent positions and the bonding curve structure
- Transaction ticker: scrolling feed of on-chain transactions with visual indicators
- Threat radar: circular overlay showing detected anomalies and prompt injection attempts
- Post-match scoreboard: detailed performance breakdown with 3D trophy ceremony
- Leaderboard: global rankings by ELO-style rating, accessible from 3D lobby
- Historical replay: full cinematic replay with timeline scrubbing and camera controls

4.4 3D Lobby & Social Features (NEW)

- Persistent 3D lobby world where players can walk around, view live arenas, and socialize
- Agent showcase room: display your NFT agents with custom lighting and poses
- War room: private spaces for team strategy planning before arena deployment
- Trophy hall: display earned achievements, past victories, and rare agent NFTs
- Emote system: player expressions and celebrations
- Voice/text chat: in-arena and lobby communication

4.5 Integration Requirements

Integration	Purpose
FunPump SDK	Token creation, AI agent deployment, bonding curve management
Meteora DLMM	Liquidity migration, concentrated pool management
Phoenix DEX	Professional market-making, MEV-protected trade execution
Crossmint	NFT minting of top-performing AI agents

Solana RPC + WebSocket	Real-time on-chain data streaming to game server
Jito (MEV)	Bundle simulation and MEV analysis
Phantom / Solflare Wallet SDK	Native wallet connection in game client (via Web Widget or C++/C# bridge)
PlayFab / GameLift (Optional)	Matchmaking, player accounts, analytics, server scaling

5. Security Requirements

5.1 AI-Specific Security

Threat	Mitigation
Prompt Injection	Input validation, instruction isolation, sandboxed LLM execution
Adversarial Inputs	ML-based input screening, tokenizer integrity verification
Model Poisoning	Federated learning with anomaly-filtered training data
Data Leakage	Differential privacy on agent strategy parameters
Model Drift	Continuous accuracy monitoring with automated retraining triggers

5.2 Blockchain Security

- Third-party smart contract audits before mainnet deployment
- Multi-signature wallet controls for protocol treasury
- Rate limiting and transaction-spam protection
- Slashing mechanisms for detected cheating or collusion
- Devnet → Testnet → Mainnet progressive deployment pipeline

5.3 Game Client Security (NEW)

- Anti-cheat integration: Easy Anti-Cheat (EAC) or BattlEye for desktop builds
- Server-authoritative game state — clients cannot modify scoring or blockchain transactions
- Encrypted communication between game client and dedicated server (TLS/DTLS)
- Code obfuscation and asset encryption to prevent reverse engineering of agent strategies
- Wallet transaction signing happens only on client-side with user confirmation — server never holds private keys
- Rate-limited API calls from game client to prevent DDoS on game servers
- Replay integrity: replays are server-recorded and cryptographically signed to prevent tampering

6. Implementation Roadmap (Updated)

Phase	Timeline	Deliverables

Phase 1 — Foundation & Smart Contracts	Weeks 1–3 (3 weeks)	Bonding curve smart contract (Anchor/Rust) with configurable parameters FunPump SDK integration & Devnet deployment Solana RPC/WebSocket event subscription service CI/CD pipeline setup & dev environment standardization
Phase 2 — AI Engine + Game Pre-Production (Parallel Tracks)	Weeks 4–7 (4 weeks)	Track A — AI Engine: ML anomaly detection models (isolation forests, autoencoders) LLM-based strategy engine & attack/defence agent templates RL training loop with sandboxed token environments Track B — Game Engine (Parallel): Engine selection finalization & project scaffolding Concept art, greybox arena blockout, hologram shader R&D Agent character model with skeleton rig & base animations Camera system prototype & HUD/UI wireframes Networking prototype: server ↔ client handshake + basic replication
Phase 3 — Game Client Alpha (Vertical Slice)	Weeks 8–13 (6 weeks)	Full arena rendering pipeline: PBR materials, dynamic lighting, post-processing Bonding curve hologram shader (wireframe + volumetric glow + data color ramp) Attack VFX (pump column, sniper beam, sandwich shockwave, swarm particles) Defence VFX (shield dome, counter-pulse ring, liquidity fountain, radar sweep) Agent characters (2 attacker + 2 defender variants) with full animation sets Wallet integration widget (Phantom/Solflare) Game server: blockchain event bridge + 30 Hz tick + delta compression HUD v1: live price overlay, agent stats, transaction ticker Internal vertical slice review & playtesting
Phase 4 — Game Client Beta & Content	Weeks 14–20 (7 weeks)	Matchmaking system: ELO-based queue, stake-tier brackets, session lifecycle Scoring engine with 3D animations & end-of-match ceremony Spectator camera: cinematic replay, timeline scrubbing, slow-motion 3D Lobby: persistent world, arena portals, agent showcase, social emotes

		Leaderboard with 3D lobby trophy cabinet Audio system: spatial SFX, adaptive music, announcer VO Full art production: 3 arena biomes, 6 agent variants, NFT cosmetic system Staking & reward UI, threat radar, mini-map overlays Performance optimization pass: LOD tuning, GPU profiling Closed beta (200–500 invited players) — feedback & analytics
Phase 5 — Integration & Security Hardening	Weeks 21–24 (4 weeks)	Meteora DLMM migration integration (on-chain + game visualization) Phoenix DEX integration for MEV-protected execution NFT agent marketplace (Crossmint) with in-game 3D preview Anti-cheat deployment (Easy Anti-Cheat / BattlEye) Asset encryption & client obfuscation Smart contract security audit (CertiK / OtterSec) Prompt injection & adversarial input hardening
Phase 6 — QA, Optimization & Pre-Launch	Weeks 25–28 (4 weeks)	Full QA regression: functional, visual, network, platform-specific Load testing: 500 spectators/arena, 50 simultaneous arenas, auto-scaling Cross-platform build validation: Windows, macOS, WebGL/Pixel Streaming Open beta (2 weeks): 2,000–5,000 players, live ops monitoring Final balance tuning: scoring weights, reward curves, matchmaking Marketing assets: cinematic trailer, gameplay footage, store pages
Phase 7 — Mainnet Launch	Weeks 29–30 (2 weeks)	Mainnet smart contract deployment with multisig governance Production game server cluster (AWS GameLift / Kubernetes) CDN setup for client distribution & asset patching Launch-day war room: real-time dashboards, hotfix pipeline Community launch event: live-streamed tournament with prize pool Post-launch stability patches & community feedback triage
Phase 8 — Post-Launch Roadmap	Weeks 31+ (Ongoing)	Mobile client (Unity URP): iOS + Android with touch controls Season 1 content: new arena biomes, agent

		skins, seasonal leaderboard Tournament system with bracket matchmaking & live spectator Community agent marketplace (encrypted strategies as NFTs) Console port feasibility (PS5/Xbox) & certification planning Cross-play infrastructure, DAO governance, public SDK

7. Success Metrics & KPIs

Metric	Target
Threat Detection Accuracy	≥ 95%
Defence Response Latency	< 2 seconds
Daily Active Arenas	100+ within 3 months of launch
Total Value Locked (Staked)	$1M+ within 6 months
Agent NFT Trading Volume	$500K+ monthly
User Retention (30-day)	≥ 40%
Smart Contract Uptime	99.9%
Game Client Frame Rate	≥ 60 FPS on recommended specs (PC)
Game Server Tick Rate	≥ 30 Hz for arena state sync
Concurrent Spectators per Arena	500+ supported
Cross-Platform DAU Split	70% PC / 20% Browser / 10% Mobile (Year 1)
Average Session Duration	≥ 25 minutes (up from 12 min in web version)

8. Technology Stack Summary (Updated)

- **GAME CLIENT**

  UE5 (C++ / Blueprints) OR Unity (C# / HDRP)
  Nanite · Lumen · Niagara (UE5) | VFX Graph · Shader Graph · Cinemachine (Unity)
  Solana Wallet Widget · REST/WebSocket Client

- **GAME SERVER**

  UE5 Dedicated Server (C++) OR Unity Netcode for GameObjects (C#)
  WebSocket Bridge · Redis Pub/Sub · gRPC · PostgreSQL
  Kubernetes / AWS GameLift for scaling

- **WEB COMPANION (Optional)**

  React / Next.js · D3.js · WebSocket Client
  Leaderboards · Account Management · NFT Marketplace Browser

- **AI ENGINE**

LLM Strategy · ML Detection · RL Evolution
Python · TensorFlow/PyTorch · LangChain

- **PROTOCOL LAYER**

  FunPump SDK · Meteora SDK · Phoenix DEX
  TypeScript · Rust

- **BLOCKCHAIN**

  Solana · Anchor Framework · SPL Tokens
  Rust

9. UE5 vs Unity Decision Matrix

Both engines are viable. The following matrix helps guide the decision based on project priorities:

Criteria	UE5	Unity	Recommendation
Visual Fidelity	Superior (Nanite, Lumen, MetaSounds)	Excellent with HDRP, slightly behind UE5	UE5 if AAA visuals are top priority
Cross-Platform	PC/Console strong; WebGL via Pixel Streaming (server cost)	Native WebGL + mobile + PC + console	Unity if browser/mobile reach is critical
Development Speed	Blueprints accelerate prototyping; C++ for performance	C# is faster to iterate; large asset store	Unity for faster iteration
Web3 Ecosystem	Growing but limited native SDKs	More mature Web3 SDKs (Thirdweb, Moralis, ChainSafe)	Unity for easier Web3 integration
Multiplayer	Built-in replication + dedicated server framework	Netcode for GameObjects + third-party options	UE5 for built-in server architecture
Team Expertise	C++ / Blueprint developers needed	C# developers (more available)	Depends on team composition
Licensing	5% royalty above $1M gross revenue	Runtime fee structure (per-install)	Evaluate based on revenue projections

10. Minimum Hardware Requirements

Component	Minimum	Recommended
OS	Windows 10 64-bit / macOS 12+	Windows 11 / macOS 14+
CPU	Intel i5-8400 / AMD Ryzen 5 2600	Intel i7-12700 / AMD Ryzen 7 5800X
GPU	NVIDIA GTX 1060 6GB / AMD RX 580	NVIDIA RTX 3070 / AMD RX 6800 XT
RAM	8 GB	16 GB
Storage	10 GB SSD	20 GB NVMe SSD
Network	10 Mbps broadband	50 Mbps broadband

Display	1080p	1440p / 4K

Appendix A: Reference Links & Developer Resources

This appendix collects official documentation, SDKs, and reference materials for all technologies introduced or updated in the UE5/Unity game platform revision. Links are organized by PRD section.

A.1 Unreal Engine 5 — Core Features (Section 3.4)

- **UE5 Download & Getting Started** Official UE5 download page and onboarding resources. https://www.unrealengine.com/download
- **Nanite (Virtualized Geometry)** Virtualized micro-polygon geometry system for film-quality detail in arenas. https://dev.epicgames.com/documentation/en-us/unreal-engine/nanite-virtualized-geometry-in-unreal-engine
- **Lumen (Global Illumination)** Fully dynamic global illumination and reflections for real-time arena lighting. https://dev.epicgames.com/documentation/en-us/unreal-engine/lumen-global-illumination-and-reflections-in-unreal-engine
- **Niagara VFX System** GPU-accelerated particle system for attack/defence visual effects. https://dev.epicgames.com/documentation/en-us/unreal-engine/niagara-visual-effects-in-unreal-engine
- **MetaSounds (Audio Engine)** Procedural audio engine for dynamic, data-driven sound design. https://dev.epicgames.com/documentation/en-us/unreal-engine/metasounds-in-unreal-engine
- **UMG (Unreal Motion Graphics) UI** In-game UI framework for HUD, dashboards, wallet controls, and analytics overlays. https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-in-unreal-engine
- **Sequencer (Cinematic Editor)** Cinematic sequencing tool for spectator replays and match highlight reels. https://dev.epicgames.com/documentation/en-us/unreal-engine/sequencer-cinematic-editor-in-unreal-engine
- **Pixel Streaming** Stream UE5 rendering to browsers — enables browser fallback without WebGL port. https://dev.epicgames.com/documentation/en-us/unreal-engine/pixel-streaming-in-unreal-engine
- **Dedicated Server Setup** Guide for building UE5 dedicated game servers for arena hosting. https://dev.epicgames.com/documentation/en-us/unreal-engine/setting-up-dedicated-servers-in-unreal-engine
- **Replication & Networking** UE5 multiplayer replication system for syncing arena state to clients. https://dev.epicgames.com/documentation/en-us/unreal-engine/networking-and-multiplayer-in-unreal-engine

A.2 Unity — Core Features (Section 3.4)

- **Unity Download & Getting Started** Official Unity Hub download and editor setup. https://unity.com/download
- **HDRP (High Definition Render Pipeline)** Console/PC-quality rendering for visually stunning arena environments. https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@latest

- **URP (Universal Render Pipeline)** Optimized pipeline for mobile and WebGL cross-platform builds.
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest
- **VFX Graph** GPU-accelerated visual effects for attack/defence particle systems.
  https://docs.unity3d.com/Packages/com.unity.visualeffectgraph@latest
- **Shader Graph** Node-based shader editor for custom bonding curve hologram materials.
  https://docs.unity3d.com/Packages/com.unity.shadergraph@latest
- **Cinemachine** Procedural camera system for spectator cameras and agent-follow cams.
  https://docs.unity3d.com/Packages/com.unity.cinemachine@latest
- **UI Toolkit** Modern UI framework for building game HUD and analytics overlays.
  https://docs.unity3d.com/Manual/UIElements.html
- **Netcode for GameObjects** Unity's first-party multiplayer networking library for arena state sync.
  https://docs-multiplayer.unity3d.com/netcode/current/about/
- **Unity WebGL Build** Browser deployment target for Web3-native users without desktop installs.
  https://docs.unity3d.com/Manual/webgl-building.html
- **Addressables (Asset Management)** Dynamic asset loading and streaming for arena maps and agent models.
  https://docs.unity3d.com/Packages/com.unity.addressables@latest

A.3 Web3 & Solana SDKs for Game Engines (Section 3.4.2 / 4.5)

Unity — Solana Integration

- **Solana.Unity-SDK (Magicblock Labs)** Native Solana SDK for Unity — wallet connection (Phantom, Solflare), transaction signing, SPL tokens, NFTs. Best fit for this project's Solana-native architecture.
  https://github.com/magicblock-labs/Solana.Unity-SDK
- **thirdweb Unity SDK** Cross-chain game SDK with wallet abstraction, smart contract interaction, NFT/token standards. GitHub: https://github.com/thirdweb-dev/unity
  https://portal.thirdweb.com/unity/v5
- **ChainSafe Gaming SDK** Open-source Unity Web3 SDK supporting multiple chains. Includes marketplace, NFT, and token modules.
  https://docs.gaming.chainsafe.io/

Unreal Engine 5 — Web3 Integration

- **Sequence Unreal Engine SDK** Embedded wallet, NFT management, gasless transactions for UE5. Includes Cropout (RTS) sample game. GitHub: https://github.com/0xsequence/sequence-unreal
  https://docs.sequence.xyz/guides/unreal-ew-guide
- **Moralis UE5 Integration** Retrieve wallet balances and NFTs via AWS Lambda backend. Sample repo: https://github.com/MoralisWeb3/demo-unreal-aws-lambda
  https://docs.moralis.com/guides/unreal-5-get-web3-data
- **VaRest Plugin (UE Marketplace)** REST/JSON API plugin for UE5 — required for Moralis and custom backend API calls.
  https://www.unrealengine.com/marketplace/en-US/product/varest-plugin

A.4 Wallet SDKs (Section 3.4.2 — Wallet Integration)

- **Phantom Wallet — Developer Docs** Primary Solana wallet. Docs cover deeplink integration, provider API, and mobile SDK.
  https://docs.phantom.app/
- **Solflare Wallet — Developer Docs** Alternative Solana wallet with browser extension and mobile support.
  https://docs.solflare.com/
- **Solana Wallet Adapter** Standard wallet connection library — wrap in WebView/browser widget for in-game use.
  https://github.com/solana-labs/wallet-adapter

A.5 Game Server & Infrastructure (Section 3.4.4)

- **AWS GameLift** Managed dedicated game server hosting with auto-scaling. Supports UE5 and Unity server builds.
  https://docs.aws.amazon.com/gamelift/
- **Azure PlayFab** Backend services for games: matchmaking, leaderboards, player accounts, analytics.
  https://learn.microsoft.com/en-us/gaming/playfab/
- **Hathora (Cloud Game Server)** Modern game server hosting with instant scaling — simpler alternative to GameLift.
  https://hathora.dev/docs
- **Agones (Kubernetes Game Servers)** Open-source, Kubernetes-native dedicated game server orchestration.
  https://agones.dev/site/docs/
- **Redis Pub/Sub** Real-time message broker between game server and blockchain event services.
  https://redis.io/docs/latest/develop/interact/pubsub/

A.6 Audio Middleware (Section 3.4.2 — Audio System)

- **FMOD for Unity** Professional audio middleware for adaptive music and spatial sound in Unity.
  https://www.fmod.com/docs/2.03/unity/
- **FMOD for Unreal** FMOD plugin for UE5 — alternative to MetaSounds for cross-engine consistency.
  https://www.fmod.com/docs/2.03/unreal/
- **Wwise (Audiokinetic)** Enterprise-grade audio middleware. Free for indie projects under revenue threshold.
  https://www.audiokinetic.com/en/products/wwise/

A.7 Anti-Cheat & Game Security (Section 5.3)

- **Easy Anti-Cheat (EOS)** Free anti-cheat from Epic Online Services. Supports UE5, Unity, and custom engines.
  https://dev.epicgames.com/docs/game-services/anti-cheat
- **BattlEye Anti-Cheat** Industry-standard proactive anti-cheat. Requires commercial license.
  https://www.battleye.com/

- **UE5 Encryption & Pak Signing** Asset encryption to prevent reverse engineering of agent strategy data.
  https://dev.epicgames.com/documentation/en-us/unreal-engine/pak-file-signing-and-encryption-in-unreal-engine

A.8 Engine Comparison Resources (Section 9)

- **UE5 Licensing & Royalty FAQ** 5% royalty above $1M gross revenue. Free for learning and sub-threshold projects.
  https://www.unrealengine.com/en-US/faq
- **Unity Runtime Fee & Pricing** Current Unity pricing tiers and runtime fee structure.
  https://unity.com/pricing
- **UE5 Migration Guide** Official migration guide for upgrading from UE4 to UE5.
  https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5-migration-guide

A.9 Reference Architectures & Tutorials

- **AI Game Master Architecture (Milehigh.world)** Decoupled architecture: Unity client + Python backend + LLM for AI-driven gameplay. Directly applicable to our AI agent orchestration pattern.
  https://medium.com/@milehigh.wrld/milehigh-world-ai-game-master-architecture-this-document-details-the-implementation-of-a-decoupled-ea3154700d6d
- **Sequence — Cropout UE5 Web3 Game (Full Sample)** Complete UE5 Web3 game project with embedded wallet, NFT management, and transactions.
  https://github.com/0xsequence/sequence-unreal
- **thirdweb — Web3 Warriors (Unity Sample)** Multiplayer on-chain survival game built with thirdweb Unity SDK.
  https://github.com/thirdweb-dev/unity
- **Solana Game Development Guide** Official Solana Foundation guide for building games on Solana — includes energy systems, NFT integration, and token gating.
  https://solana.com/developers/guides/games/getting-started-with-game-development
- **Solana Cookbook — Gaming Section** Community recipes for Solana game development patterns.
  https://solanacookbook.com/gaming/intro.html